Contents

SPL 3.0 User Guide

SPL (Structured Prompt Language) is a SQL-inspired declarative language for orchestrating LLM workflows. You write a `.spl` file — the invariant *logical view* — and the toolchain runs it, compiles it, describes it, or generates it from plain English.

> **End-to-end validated.** The ten-step pipeline documented in §15 (S1–S10) was designed for the NeurIPS 2026 paper *"Beyond Vibe Coding: Intent Invariance and Structured Prompt Language"* and executed across 5 real-world workflow recipes × 3 model tiers (15 experiments). Every command in this guide was exercised in those experiments — making §15 both the research protocol and the most comprehensive integration test suite for SPL.py.

Table of Contents

---

1. Quick-start

Provisioning a fresh machine (conda env, SageMath wheels, Lean 4 + mathlib)? Start with **SETUP.md** — this guide assumes that stack is in place.

```
# Validate a .spl file
spl3 validate cookbook/05_self_refine/self_refine.spl

# Run it with different adapters
spl3 run cookbook/05_self_refine/self_refine.spl --adapter ollama --model gemma3

# Visualise a .spl file as a Mermaid diagram (no LLM needed)
spl3 spl2mmd cookbook/05_self_refine/self_refine.spl --out-dir diagrams/

# Extract Mermaid logic from an image
spl3 img2mmd diagram.png --adapter openrouter --model google/gemini-2.0-flash-001 -o lo
```

```
# Compare two Mermaid diagrams — auto-selects GED, emits DIVERGED/DEGRADED/etc.
spl3 compare orig.mmd roundtrip.mmd --adapter claude_cli --format html -o report.html

# Compile to Python/PocketFlow (deterministic — no LLM needed)
spl3 splc compile cookbook/05_self_refine/self_refine.spl --lang python/pocketflow

# Vibe-code directly from a description (bypass IR steps)
spl3 vibe "self-refine agent that iterates until quality > 0.8" \
  --adapter claude_cli -m claude-sonnet-4-6 -o out.py
```

---

2. LLM Adapters

SPL 3.0 supports multiple LLM adapters for different deployment scenarios and cost models:

Available Adapters

Adapter	Cost Model	Authentication	Models
`claude_cli`	Subscription (flat rate)	Claude Code OAuth	`claude-sonnet-4-6` (default), any Claude model
`gemini_cli`	Free tier + subscription	Gemini CLI OAuth	`gemini-2.5-flash` (default), `gemini-3-flash-preview`
`ollama`	Local inference	None required	`gemma3` (default), `nomic-embed-text` (embedding)
`openrouter`	API usage	API Key	`google/gemini-3-flash-preview`, `qwen/qwen3.6-plus`, and many others

---

3. spl3 validate

[To-Test] Semantic linting (4.4) — new in this release.

Checks lexer → parser → semantic linter. No LLM call.

```
# Syntax + semantic checks (default)
spl3 validate workflow.spl

# Multiple files
spl3 validate tests/claude_cli/sonnet/*.spl

# Strict mode — warnings become errors (non-zero exit)
spl3 validate workflow.spl --strict

# Syntax only — skip semantic analysis
spl3 validate workflow.spl --no-semantic
```

Semantic checks (`--semantic`, default on)

Check	What it catches
Undefined variable	`@x` read before any `GENERATE … INTO @x`, `CALL … INTO @x`, or `@x :=`
Unreachable code	Statements after `RETURN` inside a workflow body
Potential infinite loop	`WHILE` with no `RETURN` inside body and no `max_iterations` declared
Undefined CALL target	`CALL proc(…)` where `proc` is not in `CREATE FUNCTION` or stdlib tools

All checks are static AST passes — no LLM involved. Issues are printed as `WARN [workflow_name] message`. Exit code is non-zero only on parse errors by default; add `--strict` to make warnings fatal too.

---

4. spl3 run

Executes a `.spl` workflow against a live LLM adapter.

```
spl3 run <file.spl> [OPTIONS]
```

4.1 Basic options

Option	Default	Description
`--adapter NAME`	`ollama`	LLM adapter (`ollama`, `claude_cli`, `openrouter`, …)

Option	Default	Description
`--model MODEL`	adapter default	Model override
`-p / --param key=val`	—	Workflow `INPUT` parameter. Repeatable.
`--tools FILE`	auto-load `tools.py`	Python module providing `@spl_tool` functions for `CALL`
`--log-prompts DIR`	off	Write each assembled prompt to `DIR/<fn>_NNN.md` before sending
`--claude-allowed-tools`	—	Comma-separated tools for the `claude_cli` adapter
`--kernel`	off	Enable persistent IPython kernel (see §4.2)
`--kernel-scope`	`session`	Kernel lifecycle: `session` or `workflow`
`--kernel-timeout`	`60.0`	Per-cell execution timeout in seconds
`--kernel-name NAME`	`python3`	Jupyter kernel spec to run kernel steps under (e.g. `sagemath`). A non-default value implies `--kernel`.
`--persistence BACKEND`	off	Enable durable execution: `sqlite` (local), `postgres` (production), `dbos` (cloud). Checkpoints every GENERATE/CALL step. See §4.3.
`--workflow-id ID`	auto-UUID	Run identifier for persistence. Pass the same ID to resume a crashed run from its last checkpoint.

```
# Basic run
spl3 run cookbook/05_self_refine/self_refine.spl --adapter ollama --model gemma3

# Pass workflow INPUT parameters
spl3 run workflow.spl --adapter ollama -p topic="linear algebra" -p depth=3

# Load a Python tools module
spl3 run workflow.spl --adapter ollama --tools mytools.py

# Log all LLM prompts for inspection
spl3 run workflow.spl --adapter claude_cli --log-prompts ./prompts/
```

4.2 IPython Kernel (`--kernel`)

`--kernel` attaches a **persistent IPython kernel** (via `jupyter_client`) to the executor session. Every `CALL run_python(@code) INTO @result` step routes through this kernel

instead of spawning a new Python subprocess.

Why it matters:

Without `--kernel`	With `--kernel`
Each `CALL run_python` is a fresh subprocess	One kernel shared across all `CALL run_python` steps
No state between calls — imports, variables lost	State accumulates: `import sympy` in step 1 still live in step 5
Only stdlib available; extra deps require `@spl_tool` wiring	Every `pip`-installed package is available — just `import` it
Subprocess output only (stdout captured as string)	Full IPython semantics: expression results, repr, stdout

Quick example:

```
# Run recipe 69 — eigenvalue notebook generator
spl3 run cookbook/69_notebook_gen/notebook_gen.spl \
    --kernel --adapter ollama \
    --tools cookbook/69_notebook_gen/tools.py
```

Alternate kernel specs (`--kernel-name`): any installed Jupyter kernel spec can back the session — e.g. `--kernel-name sagemath` runs every `CALL run_python()`, `SOLVE`, and `ASSERT` step under SageMath's Python (richer CAS: SageManifolds, GAP, PARI). If the spec is not installed, `spl3 run` fails fast with the installed-kernel list and install instructions. To install:

```
pip install 'spl-llm[sage]'                          # passagemath wheels — no source b
python -m sage.repl.ipython_kernel.install --user    # register the 'sagemath' kernel s
# or, distro-native: conda install -c conda-forge sage
```

Note the Sage kernel applies its *preparser* to cell code (`^` is power, integer literals are Sage `Integers`); emit `preparser(False)` before pure-Python verifier code. See `docs/DEV/sage_lean_integration_plan.md`.

```
spl3 run workflow.spl --kernel-name sagemath --adapter ollama
```

Lean 4 bridge (proof-checked verification): Lean is not a Jupyter kernel — it enters through the same door as everything else: a thin client *inside* the Python kernel. `spl3.lean_bridge.LeanREPL` manages a persistent `leanprover-community/repl` process (mathlib imported once and amortized; every check runs in a fresh environment forked from the warm base, with timeout → transparent restart). An SPL workflow typechecks and kernel-checks Lean statements with ordinary `CALL`/`ASSERT` steps — no new constructs:

```
-- One-time per run: start the bridge inside the kernel
CALL run_python('from spl3.lean_bridge import LeanREPL; lean = LeanREPL.mathlib().start

-- Then: statement well-formedness, kernel-checked proofs, mathlib citations
```

```
CALL run_python('ok = lean.statement_ok(stmt)') INTO @stmt_ok
CALL run_python('result = lean.check(proof_code)') INTO @proof_result
CALL run_python('cite = lean.find_citation(stmt)') INTO @mathlib_lemma
```

One-time setup: `cookbook/tools/lean/setup_lean.sh` (elan + pinned REPL build; add `--with-mathlib` for the mathlib citation path). Lean is a strictly optional dependency — workflows degrade gracefully when it is absent, and a failed proof attempt never blocks delivery: the section keeps its CAS-level badge and only the higher `machine_proved` badge (§18) is withheld.

Together these form the **verifier ladder** — numeric spot-check (NumPy) → exact symbolic instance (SymPy/Sage) → machine-checked proof (Lean 4 + mathlib) — with the same two SPL constructs (`SOLVE`/`ASSERT`) at every rung. Worked recipes:

Recipe	Demonstrates
`cookbook/75_sage_math`	Sage kernel: Galois groups, conics over ℚ, elliptic-curve rank
`cookbook/76_lean_proof`	Lean proof checking + LLM repair loop + mathlib citation path
`cookbook/71_linalg_concept_book/lean_proofs.sql`	Post-pass proving real textbook entries to `machine_proved`
`cookbook/77_neurosymbolic`	Capstone: SymPy + SageMath + Lean behind ONE workflow — the backend is a `--param backend=` flip; 29-problem battery + A/B experiment harness

Design background: `docs/DEV/sage_lean_integration_plan.md` (fully implemented).

The workflow runs three `CALL run_python` steps that share a single kernel:

```
-- Step 1: import once
CALL run_python(f'
import sympy as sp
A = sp.Matrix({@matrix_a})
print(f"A = {A}")
') INTO @matrix_repr

-- Step 2: A and sympy are still live — no re-import
CALL run_python('
eigendata = A.eigenvects()
...
') INTO @eigen_summary

-- Step 3: eigendata is still live — no re-computation
CALL run_python('
checks = [bool(A * v == lam * v) for lam, mult, vecs in eigendata for v in vecs]
...
') INTO @verification
```

Passing SPL variables into kernel code:

Use SPL f-strings with `{@variable}` for values that are safe Python literals (numbers, lists, simple strings). For LLM-generated prose, pass via a `@spl_tool` function (`tools.py`) to avoid quoting issues.

```
-- Safe: @matrix_a = "[[4, 1], [2, 3]]" becomes a valid Python literal
CALL run_python(f'A = sp.Matrix({@matrix_a})') INTO @_

-- Safe: @n is a number
CALL run_python(f'result = factorial({@n})') INTO @result

-- Best practice for prose: use tools.py, not f-string embedding
CALL assemble_notebook(@intro_text, @explanation_text, @output_path) INTO @path
```

Kernel scope options:

`--kernel-scope`	Behaviour
`session` (default)	One kernel shared across all `CALL run_python` steps in the run — imports, variables, and SymPy symbols persist end-to-end
`workflow`	Caller is responsible for restarting between workflow runs when isolation is needed (e.g. parallel batch runs)

Timeout:

`--kernel-timeout 120` sets the per-cell timeout to 120 seconds. Useful for workflows with long-running computations (large matrix factorisations, numerical optimisation). A `TimeoutError` is mapped to SPL `ToolFailed`.

Requirements:

```
pip install jupyter_client ipykernel sympy   # minimum for symbolic math
```

`jupyter_client` and `ipykernel` must be installed in the same environment as `spl-llm`. Any other package (`numpy`, `z3-solver`, `networkx`, …) is available as soon as it is `pip install`-ed — no `@spl_tool` wiring needed.

How it interacts with `--tools`:

When `--tools` is used together with `--kernel`, the stdlib's subprocess-based `run_python` (registered globally) is automatically overridden by the kernel version after tool loading. Functions from your `tools.py` that need to receive LLM-generated text should use `@spl_tool` and accept them as `str` arguments — this avoids the quoting complexity of embedding multi-line prose inside Python code strings via SPL f-strings.

Error handling:

A Python exception inside the kernel raises `KernelExecutionError`, which is mapped to SPL `ToolFailed`. The kernel recovers automatically — the session stays alive and subsequent `CALL run_python` steps continue normally.

```
KernelExecutionError: ZeroDivisionError: division by zero
→ SPL ToolFailed  →  caught by EXCEPTION handler or logged as workflow error
→ kernel session remains live for subsequent steps
```

Two kernel backends (advanced):

Backend	Class	When used
`IPythonKernel`	out-of-process via `jupyter_client`	`--kernel` flag; full IPython semantics, expression results, real notebook parity
`KernelSession`	in-process `exec()` namespace	`CREATE TOOL_API` bodies; no extra deps, captures stdout only

`--kernel` always uses `IPythonKernel`. `KernelSession` is used internally for `CREATE TOOL_API` definitions and does not require `jupyter_client`.

4.3 Durable Execution (`--persistence`)

By default SPL workflows are stateless across crashes: if a 50-step research agent fails at step 43, you restart from zero. `--persistence` enables **crash recovery and exactly-once execution** — completed GENERATE and CALL steps are never re-run.

Key principle (DODA): the `.spl` file never changes. Persistence is a physical decision resolved at runtime via flags, exactly like `--adapter` selects the LLM provider.

```
# Stateless (default)
spl3 run workflow.spl --adapter ollama -m gemma3

# Durable local — SQLite, zero extra dependencies
spl3 run workflow.spl --adapter ollama -m gemma3 --persistence sqlite

# Durable production — PostgreSQL (pip install asyncpg + SPL_PG_DSN env var)
spl3 run workflow.spl --adapter ollama -m gemma3 --persistence postgres

# Durable cloud — DBOS (pip install dbos + DBOS_DATABASE_URL)
spl3 run workflow.spl --adapter ollama -m gemma3 --persistence dbos \
  --workflow-id my-run-001

# Resume a crashed run — pass the same workflow-id
spl3 run workflow.spl --adapter ollama -m gemma3 --persistence sqlite \
  --workflow-id my-run-001
```

When `--workflow-id` is omitted a UUID is auto-generated and printed, so you can pass it back on resume.

Backends at a glance

Backend	When to use	Setup
`sqlite`	Local dev, single-node	Zero deps — DB at `~/.spl/workflows.db`
`postgres`	Production, multi-node	`pip install asyncpg` + `export SPL_PG_DSN=postgresql://...`
`dbos`	Cloud-grade, HITL dashboard	`pip install dbos` + `export DBOS_DATABASE_URL=postgresql:/`

All three backends implement the same `PersistenceBackend` interface — swap backends without touching the `.spl` file or the executor.

What gets checkpointed Only the two non-idempotent operation types are checkpointed:

SPL statement	Checkpointed?	Why
`GENERATE … INTO @var`	✓	LLM call — expensive and non-deterministic
`CALL tool() INTO @var`	✓	Tool execution — may have side effects
`@var := expression`	—	Cheap, deterministic; re-derived from saved state
`EVALUATE … WHEN`	—	Pure conditional on already-checkpointed vars
`WHILE condition DO`	—	Loop guard is deterministic given saved state

Each checkpoint stores the full `@var` snapshot, so on resume the executor restores the complete workflow state from the last successful step and continues.

HITL — Human-in-the-Loop No new SPL keywords are needed. Approval gates use standard `CALL`:

```
WORKFLOW review_and_publish
  INPUT @draft TEXT, @workflow_id TEXT
  OUTPUT @published TEXT
DO
  GENERATE edit_draft(@draft) INTO @polished;

  -- Suspend indefinitely until a human approves (survives restarts)
  CALL wait_for_approval(@workflow_id, "approve_draft") INTO @decision;

  EVALUATE @decision
    WHEN contains("approved") THEN
      CALL publish(@polished) INTO @published;
    ELSE
      @published := "rejected";
  END;

  RETURN @published WITH status = "complete";
END;
```

`wait_for_approval` and `send_approval` are automatically registered as CALL targets when `--persistence` is active. An external system (CLI, webhook, dashboard) unblocks the waiting workflow:

```
spl3 workflow send-event \
  --workflow-id my-run-001 \
  --key approve_draft \
  --value "approved"
```

```
# List all durable runs
spl3 workflow list
spl3 workflow list --status running

# Show completed steps and current @var state
spl3 workflow status --workflow-id my-run-001

# Send a HITL event to unblock a waiting workflow
spl3 workflow send-event --workflow-id my-run-001 --key approve_draft --value approved

# Resume a crashed workflow from last checkpoint
spl3 workflow resume workflow.spl --workflow-id my-run-001 --adapter ollama
```

`spl3 workflow` subcommand

Design notes

- **Global step counter:** parent and sub-workflows (`CALL workflow_name()`) share one executor instance. The step counter is global across all nested calls — no namespace collisions between sub-workflow steps.
- **Top-level only lifecycle:** `start_workflow` / `finish_workflow` are called only once at the top-level entry point, not recursively for each sub-workflow CALL.
- **`PersistenceBackend` as shared contract:** the abstract interface is the contract between SPL.py and Momagrid — both codebases implement the same six methods (`start_workflow`, `get_step_result`, `checkpoint`, `finish_workflow`, `wait_for_event`, `send_event`). A Momagrid-native backend can be swapped in with `--persistence momagrid` without any other changes.
- **PostgreSQL HITL:** the `postgres` backend uses LISTEN/NOTIFY for sub-millisecond event delivery — no polling. `send_event` calls `pg_notify(channel, value)` so any suspended `wait_for_event` wakes up immediately.

Full design document: `docs/DEV/spl3-state-management-dbos.md`.

---

5. spl3 describe

Generates a plain-English functional specification from `.spl` source.

```
spl3 describe cookbook/05_self_refine/self_refine.spl --adapter claude_cli
```

---

6. spl3 text2spl

Compiles a natural language description into valid SPL 3.0 source code.

```
spl3 text2spl "build a review agent that refines text until quality > 0.8" \
  --mode workflow -o review.spl
```

---

7. spl3 text2mmd

Converts natural language workflow descriptions into Mermaid flowchart diagrams using an LLM. The generated `.mmd` file can be reviewed and edited before converting to SPL with `mmd2spl`.

```
spl3 text2mmd "user onboarding workflow with approval gates"
```

File input and section extraction

When the argument is a file path, `text2mmd` pre-processes the content before sending it to the LLM:

1. **Named sections** — if `## Summary` and/or `### 1. Purpose` exist, only those sections are used as the prompt. The rest of the file is ignored.

2. **Numbered sections** — if `## 0. ...` / `## 1. ...` headings exist (e.g. from `splc describe`), sections 0 and 1 are extracted.
3. **Fallback** — if neither convention is found, the LLM is asked to produce a concise 3–6 sentence workflow description from the full file, and that summary becomes the prompt.

This keeps the Mermaid prompt focused on intent rather than implementation detail, and avoids context-window overload from large spec files.

```
# File input: only ## Summary and ### 1. Purpose sent to LLM
spl3 text2mmd S1-agent-spec.md --adapter claude_cli -o S2-agent.mmd

# Preview what will be sent to the LLM (--prompt exits before any API call)
spl3 text2mmd S1-agent-spec.md --adapter claude_cli --prompt
```

---

8. spl3 img2mmd

Analyzes an image with a multimodal LLM and reconstructs the workflow or diagram as valid **Mermaid flowchart code** (`.mmd`).

Handles flowcharts, architecture diagrams, hand-drawn sketches, and pseudo-code diagrams. When no diagram structure is detected (e.g. a regular photo), it returns `(No workflow logic detected)` instead of writing a file.

How it works

1. The image is encoded as base64 and sent to the vision LLM alongside a structured prompt with explicit Mermaid syntax rules.
2. The LLM returns a `flowchart TD` block.
3. The output is extracted from any markdown fences and written to a `.mmd` file.

Options

Option	Default	Description
`--adapter`	`openrouter`	Multimodal adapter — requires `OPENROUTER_API_KEY`
`--model`	adapter default	Model override, e.g. `google/gemini-2.0-flash-001`
`-o / --out`	—	Output file path (or directory)
`--out-dir`	—	Output directory; filename derived from image stem

Examples

```
# Extract workflow from a pre-generated Mermaid PNG (round-trip verification)
spl3 img2mmd \
  /home/wengong/projects/digital-duck/Cookbook-of-SPL-Recipes/drafts/book-v0.5/mermaid/
  --out-dir /tmp/spl3 \
  --adapter openrouter \
  --model google/gemini-2.0-flash-001

# Try on an unrelated photo — returns "(No workflow logic detected)"
spl3 img2mmd /home/wengong/Pictures/china-1.png \
  --out-dir /tmp/spl3 \
  --adapter openrouter \
  --model google/gemini-2.0-flash-001

# Save to a specific file
spl3 img2mmd whiteboard.jpg -o logic.mmd --adapter openrouter
```

Adapter notes

Adapter	Requirement	Notes
`openrouter`	`OPENROUTER_API_KEY`	Default; routes to any vision model on OpenRouter
`google`	`GOOGLE_API_KEY`	Gemini models
`anthropic`	`ANTHROPIC_API_KEY`	Claude 3+ models via Anthropic API
`claude_cli`	`ANTHROPIC_API_KEY`	Uses Anthropic SDK directly for vision (CLI has no image flag)

Round-trip use case

`img2mmd` is the reverse of `spl2mmd`. After generating a PNG diagram you can verify the LLM can reconstruct the original logic from the visual:

```
spl3 spl2mmd workflow.spl --out-dir diagrams/ --no-preview
spl3 img2mmd diagrams/workflow.png -o roundtrip.mmd
spl3 compare workflow.mmd roundtrip.mmd --mode ged
```

---

9. spl3 img2text

Extracts **all text, pseudo-code, and code fragments** from a screenshot or image using a multimodal LLM (OCR + structure preservation).

Preserves indentation and layout. Detected code is wrapped in fenced code blocks with inferred language tags. Mathematical notation is rendered in LaTeX or plain ASCII.

Options

Option	Default	Description
`--adapter`	`openrouter`	Multimodal adapter — requires `OPENROUTER_API_KEY`
`--model`	adapter default	Model override, e.g. `google/gemini-2.0-flash-001`
`-o / --out`	—	Output file path (or directory)
`--out-dir`	—	Output directory; filename derived from image stem

Examples

```
# Extract text from a Mermaid diagram PNG (reads node labels, edge labels, subgraph tit
spl3 img2text \
  /home/wengong/projects/digital-duck/Cookbook-of-SPL-Recipes/drafts/book-v0.5/mermaid/
  --out-dir /tmp/spl3 \
  --adapter openrouter \
  --model google/gemini-2.0-flash-001

# Extract text from any image (photo, screenshot, whiteboard)
spl3 img2text /home/wengong/Pictures/china-1.png \
  --out-dir /tmp/spl3 \
  --adapter openrouter \
  --model google/gemini-2.0-flash-001

# Extract pseudo-code from a screenshot, save to a specific file
spl3 img2text pseudocode.png -o extracted.txt --adapter openrouter
```

Typical use cases

- **Pseudo-code on a whiteboard or slide** → extract to `.txt`, then pipe to `spl3 text2spl`
- **Screenshot of a code snippet** → extract preserving indentation and language
- **Diagram with labels** → extract all node text and annotations for review
- **Scanned document** → OCR with structure preservation

---

10. spl3 spl2mmd

Generates a Mermaid flowchart **directly from a `.spl` file's AST** — no LLM required. Every workflow, procedure, loop, branch, and exception handler is rendered deter-

ministically from the parsed syntax tree, producing a diagram that faithfully mirrors the actual script structure.

Use it to **visually review or validate** a `.spl` file at any point in the pipeline: after `mmd2spl` (S3) to verify the generated SPL matches the original diagram, when auditing a hand-written workflow, or to produce publication figures.

```
spl3 spl2mmd workflow.spl                              # defaults: .mmd + .html + .md +
spl3 spl2mmd *.spl --out-dir diagrams/                 # batch convert, all outputs in d
spl3 spl2mmd workflow.spl --no-preview                 # suppress browser auto-open
spl3 spl2mmd workflow.spl --save-pdf --out-dir diagrams/  # add PDF for publication
spl3 spl2mmd step1.spl step2.spl --save-pdf --save-png --out-dir diagrams/
```

Node shapes by statement type

Shape	Mermaid syntax	SPL statement
Parallelogram	`[/…/]`	`GENERATE … INTO` (LLM call)
Subroutine	`[[…]]`	`CALL procedure()`
Diamond	`{…}`	`WHILE`, `EVALUATE`, exception handler
Cylinder	`[(…)]`	`STORE`, storage-assign
Stadium	`([…])`	Start, End, `RETURN`, `RAISE`
Flag	`>…]`	`LOGGING`
Rectangle	`[…]`	Assignment, `SELECT INTO`, generic

Each workflow/procedure is wrapped in its own labelled subgraph. Nodes are colour-coded by type: **blue** = LLM call, **amber** = procedure call, **purple** = control flow, **green** = storage, **pink** = terminal, **grey** = log.

Options

Option	Default	Description
`--out-dir DIR`	beside each input	All outputs written here; source `.spl` copied too
`--preview` / `--no-preview`	**on**	Open `.html` in the browser immediately
`--save-html` / `--no-save-html`	**on**	Write a browser-viewable `.html`
`--save-markdown` / `--no-save-markdown`	**on**	Write a `.md` with fenced mermaid block
`--save-spl` / `--no-save-spl`	**on**	Copy source `.spl` into `--out-dir`
`--save-png`	off	Write `.png` via headless Chrome/Chromium
`--save-pdf`	off	Write `.pdf` — tries `mmdc` first, then Chrome headless

Output files (example with `--out-dir diagrams/`)

```
diagrams/
  self_refine.mmd      ← raw Mermaid source (edit with any Mermaid tool)
  self_refine.md     ← Markdown with fenced diagram (VS Code / GitHub preview)
  self_refine.html     ← standalone browser page with live rendering
  self_refine.spl      ← copy of the source script
  self_refine.pdf    ← (--save-pdf) print-ready, A4 landscape, neutral theme
  self_refine.png      ← (--save-png) rasterised full-page screenshot
```

mmdc setup (recommended for `--save-pdf`)

mmdc is the official Mermaid CLI. When present, it produces a **native vector PDF** — the highest-quality output for paper figures, posters, and slides. If `mmdc` is not found, `spl2mmd` falls back to Chrome headless `--print-to-pdf` (A4 landscape, neutral theme, no headers).

```
# Install mmdc globally (requires Node.js ≥ 18)
npm install -g @mermaid-js/mermaid-cli

# Verify installation
mmdc --version

# What spl2mmd calls internally for PDF
mmdc -i workflow.mmd -o workflow.pdf -t neutral -b white
```

> **Note:** `mmdc` downloads Chromium on first install (~200 MB via Puppeteer). On headless CI/CD servers, supply a `puppeteerConfigFile` pointing to an existing Chrome installation to avoid the download:
>
> ```
> mmdc -i workflow.mmd -o workflow.pdf \
>   --puppeteerConfigFile /etc/mmdc-puppeteer.json
> # /etc/mmdc-puppeteer.json: { "executablePath": "/usr/bin/google-chrome" }
> ```

Multi-file projects

When workflows `CALL` each other across files, run `spl2mmd` on all files together. Each produces its own diagram; cross-file calls appear as `CALL` subroutine nodes labelled with the procedure name — the callee is not inlined, keeping each diagram focused on a single file's logic.

```
spl3 spl2mmd step1.spl step2.spl step3.spl --out-dir diagrams/ --save-pdf
# or with glob
spl3 spl2mmd *.spl --out-dir diagrams/ --save-pdf
```

---

11. spl3 mmd2spl

Converts a Mermaid flowchart diagram into executable SPL workflow code.

```
spl3 mmd2spl workflow.mmd --adapter claude_cli -o workflow.spl
```

---

12. spl3 compare

A **multi-tier diff tool** that automatically selects the most appropriate comparison algorithm for each file type, then synthesises all tier results into a single verdict:

EQUIVALENT · **REFACTORED** · **DEGRADED** · **DIVERGED**

The synthesis layer is not a summary — it reasons *across* tiers: agreements confirm confidence; contradictions expose subtler patterns (e.g. "same vocabulary, different control flow" = REFACTORED even when character-level diff is large).

```
spl3 compare <file1> <file2> [OPTIONS]
```

Six comparison tiers

#	Tier	Mode	Auto-default for	What it measures
1	Topological	`ged`	`.mmd`	SPL-node-aware Graph Edit Distance
2	Semantic	`llm` / `vision`	`.md`, `.spl` / images	Intent, meaning, logical purpose
3	Syntactic	`ast-diff`	(manual)	AST-level inventory: GENERATE/CALL/workflow names
4	Structural	`structural`	(manual)	Document skeleton: headings, class/function names
5	Character	`git-diff`	`.py`, `.js`, `.ts`	Line-level text diff
6	Embedding	`vector` / `bert-score`	(manual)	Cosine similarity in embedding space

All modes are additive and composable. The default tier is auto-selected from the file extension; override or add tiers with `--mode`.

File-type auto-dispatch

```
spl3 compare a.mmd b.mmd          # → ged  (no LLM needed)
spl3 compare a.spl b.spl          # → llm
spl3 compare a.md  b.md           # → llm
spl3 compare a.py  b.py           # → git-diff
spl3 compare a.png b.png          # → vision (PIL pixel-diff + LLM vision if available)
```

Synthesis verdict

When `--synthesize` is on (default) and `--adapter` is set, a final LLM pass integrates all tier results into a single structured verdict:

Verdict	Meaning
`EQUIVALENT`	Functionally and semantically the same (surface noise allowed)
`REFACTORED`	Structure/intent preserved; presentation or style changed
`DEGRADED`	One file is a lossy version of the other — information was lost
`DIVERGED`	Genuinely different intent, logic, or purpose

When no LLM adapter is available, a **rule-based fallback** drives the verdict from GED normalized distance alone (0 → EQUIVALENT, < 0.10 → REFACTORED, < 0.35 → DEGRADED, else → DIVERGED).

LLM fallback for failed tiers

If a mode-specific tier fails (e.g. `networkx` not installed for `ged`, or `dd-embed` not installed for `vector`) and `--adapter` is set, `compare` automatically runs a targeted LLM analysis covering the failed tier's aspects rather than leaving it blank.

Options

Option	Default	Description
`--mode MODE`	auto (by ext)	Tier(s) to run. Repeatable. Choices: `llm` `git-diff` `vector` `bert-score` `ged` `vision` `ast-diff` `structural`
`--adapter NAME`	`ollama`	LLM adapter for all analysis: semantic, vision, synthesis, fallback
`--model MODEL`	adapter default	Model override
`--adapter-embed NAME`	same as `--adapter`	Adapter for embedding modes (vector, bert-score)
`--adapter-synthesis NAME`	same as `--adapter`	Adapter for synthesis pass
`--format`	`markdown`	Output format: `markdown` · `json` · `text` · `html`
`--focus`	`all`	LLM focus: `all` · `structure` · `logic` · `quality` · `syntax` · `spl`

Option	Default	Description
`--diff-style`	`unified`	git-diff style: `unified` · `context` · `side-by-side`
`--synthesize / --no-synthesize`	on	Run synthesis LLM pass to produce verdict
`--output / -o FILE`	stdout	Write report to file
`--prompt`	off	Print LLM prompt and exit (mode=llm)

> **`--focus spl`** activates a domain-aware prompt that treats `GENERATE` function signatures as the semantic heart, `EXCEPTION` handlers as safety contracts, and `CALL` dependencies as primary evidence — rather than treating `.spl` as generic text.

Output formats

`--format markdown` (default) — renders a structured report with the synthesis verdict banner at the top, followed by collapsible tier sections. Suitable for saving as `S6-*.md` or `S10-*.md` pipeline artifacts.

`--format html` — renders a **3-panel side-by-side report**: - Top-left panel: File 1 (Mermaid diagrams rendered live via Mermaid JS; images embedded; code shown as-is) - Top-right panel: File 2 (same) - Bottom panel: synthesis verdict (colour-coded banner) + collapsible tier details

`--format json` — machine-readable, includes all tier results and metadata. Useful for programmatic processing or aggregating results across experiments.

`--format text` — compact terminal output, verdict first.

Examples

```
# .mmd round-trip check — GED auto-selected, HTML report with live diagrams
spl3 compare orig.mmd roundtrip.mmd \
  --adapter claude_cli --format html -o roundtrip.html

# .spl semantic comparison with SPL-domain prompt
spl3 compare v1.spl v2.spl \
  --adapter claude_cli --focus spl

# Multi-tier: topology + semantics + syntax (for deep SPL diff)
spl3 compare a.spl b.spl \
  --mode llm --mode ast-diff --mode git-diff \
  --adapter claude_cli --focus spl --format html -o diff.html

# Spec round-trip fidelity — S6 in the NDD pipeline
spl3 compare S1-spec.md S5-spec.md \
  --adapter claude_cli --model claude-opus-4-6 \
```

```
  -o S6-spec-diff.md

# Compare two Python files with LLM-assisted logical diff (upgrade from git-diff)
spl3 compare old.py new.py --mode llm --mode git-diff --adapter claude_cli

# Image comparison (PNG Mermaid diagrams)
spl3 compare orig.png roundtrip.png --mode vision --adapter claude_cli

# No LLM — pure GED with rule-based verdict
spl3 compare a.mmd b.mmd --adapter "" --format text

# Full ablation comparison — S10 in the NDD pipeline
spl3 compare S6-ir-diff.md S9-vibe-diff.md \
  --adapter claude_cli --model claude-opus-4-6 \
  -o S10-ablation.md

# ROUGE score — fast deterministic n-gram overlap (no LLM, no model download)  [To-Test
spl3 compare S1-spec.md S5-spec.md --mode rouge
# Requires: pip install rouge-score
```

`--mode rouge` — ROUGE score

[To-Test] New in this release (3.1).

ROUGE-1, ROUGE-2, and ROUGE-L measure n-gram overlap between two documents. Faster and lighter than `llm` or `bert-score` — fully deterministic, no API call. Suitable as a quick baseline check on spec similarity before running LLM comparison.

```
spl3 compare spec1.md spec2.md --mode rouge
```

Output (markdown):

```
| Metric    | Precision | Recall | F1     |
|-----------|-----------|--------|--------|
| ROUGE-1   | 0.7200    | 0.6800 | 0.6994 |
| ROUGE-2   | 0.4100    | 0.3900 | 0.3998 |
| ROUGE-L   | 0.6500    | 0.6200 | 0.6347 |
```

Optional dependency: `pip install rouge-score`.

Round-trip consistency workflow

`spl3 compare` is the natural quality gate for the `spl2mmd → mmd2spl → spl2mmd` round-trip. Compare the original `.mmd` against the re-generated `.mmd` to detect what `mmd2spl` lost:

```
# Generate original diagram (deterministic)
spl3 spl2mmd workflow.spl --out-dir diagrams/ --no-preview
```

```
# Round-trip: Mermaid → SPL → Mermaid
spl3 mmd2spl diagrams/workflow.mmd --adapter claude_cli -o roundtrip.spl
spl3 spl2mmd roundtrip.spl --out-dir roundtrip/ --no-preview

# Semantic diff: topology (GED) + synthesis verdict
spl3 compare diagrams/workflow.mmd roundtrip/roundtrip.mmd \
  --adapter claude_cli --format html -o roundtrip-report.html
```

A `DEGRADED` verdict identifies specific lost nodes; `EQUIVALENT` confirms lossless round-trip. The HTML report renders both diagrams side-by-side for direct visual comparison.

---

13. spl3 judge

Evaluates a file against a named rubric using an LLM judge. Returns a structured verdict — **PASS / FAIL / ESCALATE** — with per-criterion scores, chain-of-thought reasoning, and actionable feedback.

`spl3 judge` answers *"how good is this?"* against explicit criteria. `spl3 compare` answers *"how different are these two?"*. The two commands are orthogonal and complementary.

```
spl3 judge <file> [OPTIONS]
```

`--llm` convention

Judge specs use the format `ADAPTER:MODEL-ID`. The model-id may contain `/` (as OpenRouter models do) — the colon is always the adapter/model delimiter:

```
claude_cli:claude-opus-4-6
openrouter:google/gemini-2.5-pro
openrouter:qwen/qwen-max
ollama:gemma3
```

`--llm` wins over the legacy `--adapter` / `--model` flags if both are given.

Options

Option	Default	Description
`--criteria BUILTIN|FILE`	`clarity`	Built-in rubric name or path to a `.yaml` rubric
`--llm ADAPTER:MODEL`	—	Judge spec. **Repeat for panel mode** (judges run concurrently). Wins over `--adapter`/`--model`.
`--adapter NAME`	`ollama`	Legacy: judge adapter (ignored when `--llm` is given)

Option	Default	Description
`--model MODEL`	adapter default	Legacy: judge model (ignored when `--llm` is given)
`--aggregation`	`majority`	Panel aggregation: `majority` · `confidence_weighted` · `unanimous`
`--swap-check`	off	Re-run with reversed criterion order; flag if verdict disagrees
`--format`	`markdown`	Output format: `markdown` · `json` · `text`
`-o / --output FILE`	stdout	Write report to file
`--prompt`	off	Print the judge prompt and exit (no LLM call)
`--cache-key KEY`	—	Layer 2 cache integration: on a **PASS** verdict, add the `ai_reviewed` exposition badge to that content-cache entry (the `JudgeResult` is stored alongside). FAIL/ESCALATE leave the entry untouched. Key is the hex digest from `spl3 cache list`. See §18

Built-in rubrics

Name	Criteria	Use case
`clarity`	prose_clarity, completeness, appropriate_detail, no_ambiguity	Prose quality
`correctness`	factual_accuracy, mathematical_correctness, logical_consistency, no_contradictions	Math / factual content
`spl-compliance`	workflow_identity, generate_signatures, call_dependencies, evaluate_conditions, exception_handlers, variable_data_flow	SPL source review
`ai-review`	helpfulness, harmlessness, honesty, task_completion	General AI output quality

Single judge

```
# Evaluate prose clarity
spl3 judge output.md --criteria clarity --llm claude_cli:claude-opus-4-6
```

```
# Evaluate mathematical correctness, save report
spl3 judge my_section.md --criteria correctness \
    --llm openrouter:google/gemini-2.5-pro -o judge-report.md

# Evaluate SPL compliance (NDD pipeline S6 replacement/augmentation)
spl3 judge S5-spec.md --criteria spl-compliance \
    --llm claude_cli:claude-opus-4-6 -o S6-judge.md

# Legacy adapter/model style (still works)
spl3 judge output.md --criteria clarity --adapter ollama --model llama3

# Preview the judge prompt without calling the LLM
spl3 judge output.md --criteria clarity --llm claude_cli:claude-opus-4-6 --prompt

# Review a cached textbook section; on PASS the entry gains the ai_reviewed badge
spl3 judge section.md --criteria correctness --llm ollama:llama3.2 --cache-key <hex>
```

Panel mode

Repeat `--llm` to run multiple judges concurrently. Results are aggregated via `--aggregation`.

```
# Majority vote — 3 judges, concurrent
spl3 judge output.md --criteria spl-compliance \
    --llm claude_cli:claude-opus-4-6 \
    --llm openrouter:google/gemini-2.5-pro \
    --llm openrouter:qwen/qwen-max \
    --aggregation majority --swap-check

# Unanimous — PASS only if all agree (conservative quality gate)
spl3 judge output.md --criteria correctness \
    --llm claude_cli:claude-opus-4-6 \
    --llm openrouter:google/gemini-2.5-pro \
    --aggregation unanimous -o panel-report.md
```

Aggregation strategies:

Strategy	Verdict rule	Score	When to use
`majority`	PASS if > 50% PASS; tied → ESCALATE	Mean	Default; balanced
`confidence_weighted`	Weighted by each judge's self-reported confidence	Weighted mean	When judge quality varies
`unanimous`	PASS only if all PASS; else ESCALATE	Min (conservative)	High-stakes quality gates

A **SPLIT** (evenly divided panel) always yields **ESCALATE** rather than a coin flip —

surfacing the disagreement for human review.

Custom rubric

```
# YAML file: name, criteria, pass_threshold, weight (optional), prompt_template (optior
spl3 judge output.md --criteria ./rubrics/my-domain.yaml --llm claude_cli:claude-opus-4
```

Rubric YAML format:

```yaml
name: my-domain
criteria:
  - accuracy
  - conciseness
  - domain_coverage
pass_threshold: 7.0
weight:
  accuracy: 0.5
  conciseness: 0.2
  domain_coverage: 0.3
prompt_template: |
  Focus on domain-specific terminology and technical accuracy.
```

Output formats

`--format markdown` (default) — verdict banner, criteria scores table, reasoning, and feedback. Suitable for pipeline artifacts (`S6-judge.md`).

`--format json` — machine-readable; includes all fields. For single judge: `verdict`, `score`, `confidence`, `criteria_scores`, `reasoning`, `feedback`, `swap_consistent`. For panel: adds `consensus`, `aggregation`, `individual[]`, `dissent`.

`--format text` — compact terminal output.

Swap-consistency check (`--swap-check`)

Runs the judge a second time with criterion order reversed. If the verdict differs, `swap_consistent: false` is flagged in the report — indicating the result may be sensitive to position bias. For panels, `swap_consistent` is `true` only if every individual judge passed the check.

---

14. spl3 splc compile

Deterministic or LLM-assisted compilation of a `.spl` logical view to a physical target.

```
# Deterministic — Python/PocketFlow (no LLM needed)
spl3 splc compile agent.spl --lang python/pocketflow

# LLM-assisted — any target
```

```
spl3 splc compile agent.spl --lang go --llm --adapter claude_cli
spl3 splc compile agent.spl --lang python/crewai --llm --adapter openrouter -m qwen/qwe

# Domain notebook under the SageMath kernel — the .ipynb declares kernelspec 'sagemath'
# (see §4.2 for installing the Sage kernel; docs/DEV/spl3-sagemath.md for details)
spl3 splc compile build_concept_book.spl --lang python/domain_textbook --kernel-name sa
```

Supported targets

`--lang`	Transpiler	Notes
`python/pocketflow`	Deterministic	Default for NDD experiments
`python/langgraph`	Deterministic	
`python/linalg`	Deterministic	Domain target — compiles `SOLVE`/`ASSERT` graph + verification workflows to a runnable Jupyter notebook (DODA: no LLM call needed to compile). See recipe 71 and **Style profiles** below
`go`	Deterministic	
`ts`	Deterministic	
`python/crewai`	LLM only	Requires `--llm`
`python/autogen`	LLM only	Requires `--llm`
`python`	LLM only	Plain Python, requires `--llm`

Style profiles (`python/linalg` and other domain targets)

Domain targets such as `python/linalg` compile workflows that take a `@style` runtime `INPUT` and resolve it to a **prose** profile — tone, depth, audience, length, structure — *without changing what is taught or whether it's correct*. `SOLVE @style_guide TEXT := style_instruction(@style)` resolves the chosen profile to an instruction block once, and that block is threaded into every `GENERATE` prompt as `{style_guide}`. The symbolic checks (`verify_math`, `shape_check`, `reducible`, ...) never see `@style` and never change their verdict because of it — **style is orthogonal to truth**: one verified body of mathematics, five very different reading experiences.

`cookbook/71_linalg_concept_book/style_profiles.py` ships five profiles:

Style	Audience	Tone	Structure
`textbook`	first-year university student (calculus background)	precise and formal	Definition → Worked example → Key theorem → Lab cell (SymPy)
`feynman`	curious person comfortable with high-school algebra; no university math	intuitive, story-driven — build intuition, then let the algebra follow inevitably	Motivating story → Intuition → Minimal formalisation → "Now you try"
`flashcard`	student reviewing the night before an exam	terse, precise, exam-ready — one fact per card, no proofs	Q: [precise question] → A: [minimal complete answer] → Example: [one line]
`instructor`	instructor preparing a lecture or recitation	pedagogical — explicitly names common misconceptions and how to counter them	Concept summary → Common mistakes → Teaching tip → Suggested exercise
`research`	grad student / researcher who needs a precise, citable statement	dense and formal, theorem-proof style, citation-ready	Definition → Theorem → Proof → Remark (connections / generalisations)

Each profile also fixes a target length per section (e.g. `textbook`: 300–400 words, `flashcard`: 50–100 words / one Q&A pair, `instructor`: 400–500 words) — part of the same instruction block, so the LLM is told not just *how* to write but *how much*.

`@style` is a **runtime** workflow input, not a compile-time constant — the compiled notebook's structure (cell count, cell order, verification cells) is identical regardless of which style is selected; only the generated prose differs:

```
spl3 run cookbook/71_linalg_concept_book/build_concept_book.spl \
    --kernel --adapter ollama --model gemma3 \
    -p style=feynman -p target=spectral_theorem
```

See cookbook recipe 71 for the full pipeline: concept graph → productivity-ordered curriculum → style-guided generation → symbolic verification (SymPy) → notebook compilation.

Inspecting, sharing, and composing concept graphs (`scripts/concept_graph.py`)

Every concept-book domain (e.g. `linalg_graph.py` for recipe 71, `geometry_graph.py` for recipe 73) is a self-contained Python module that exposes `build() -> networkx.DiGraph` — the same graph the `productivity_order` / `reducible` / `learning_path` checks above operate on. `scripts/concept_graph.py` is a generic CLI that works against *any* such module purely through that `build()` contract — no domain module imports it or knows it exists, so two domains' notions of e.g. "ancestors" can never silently diverge (each domain keeps its own copy of the graph algorithms; see cookbook/70's readme for why that duplication is intentional). It is the one *reporting and sharing* layer every domain gets "mixed in" for free:

```
# Inspect — stats, list, show, ancestors, productivity order, learning paths
python scripts/concept_graph.py --domain linalg_graph stats
python scripts/concept_graph.py --domain linalg_graph show spectral_theorem
python scripts/concept_graph.py --domain linalg_graph path spectral_theorem -k norm

# Visualize — mermaid / graphviz-dot / ascii outline (no plotting libs needed)
python scripts/concept_graph.py --domain linalg_graph visualize --format mermaid -o gra

# Share a domain (or just a target's ancestor closure) as portable JSON
python scripts/concept_graph.py --domain linalg_graph export shared.json -t pca
python scripts/concept_graph.py import shared.json

# Compose a hybrid, multi-domain concept graph
python scripts/concept_graph.py compose -d linalg_graph -d geometry_graph hybrid.json
python scripts/concept_graph.py --domain hybrid.json stats
```

`--domain` accepts an importable module name, a path to a `.py` domain module, or a path to a `.json` graph written by `export`/`compose` — so exported and composed graphs flow straight back through the same CLI. `compose` is the framework's first mechanical step toward authoring concept graphs that span multiple fields — exactly the kind of "too daunting to publish by hand" task the concept-book framework exists to make tractable. See `scripts/README-concept_graph.md` for the full command reference.

---

15. spl3 splc describe

Reverse-engineers a specification from a **compiled target implementation**. Used in step S5 of the NDD round-trip pipeline.

```
spl3 splc describe targets/python_pocketflow/agent.py --adapter claude_cli
spl3 splc describe targets/python_pocketflow/ --adapter openrouter -m qwen/qwen3.6-plus
```

The generated spec feeds back into the reverse pipeline:

```
splc describe → spec.md → text2spl → .spl → splc compile → new target
```

---

16. spl3 vibe

One-shot prototype generator: **NL description → working code + README + test data**, in a single LLM call. No `.mmd` or `.spl` IR steps required.

Works with any model available via `ollama` (local), `claude_cli`, or `openrouter` (400+ cloud models). The `--out-dir` mode is recommended — it writes all three outputs to a folder so `spl3 splc describe` can process them as a unit in step S8.

```
spl3 vibe "<description>" [OPTIONS]
# or
spl3 vibe --description <TEXT_OR_FILE> [OPTIONS]
# or (spec-driven mode)
spl3 vibe --spec <SPEC_FILE> [OPTIONS]
```

Input modes

`vibe` supports two mutually exclusive input modes that differ in how the spec is consumed before being sent to the LLM:

Mode	Option	Behaviour
Description	positional / `--description`	Free text or file. If a file, extracts `## Summary` / `### 1. Purpose` sections (or numbered sections 0–1, or LLM-summarizes as fallback). Prompt header: `# Requirement to Implement`.
Spec-driven	`--spec SPEC_FILE`	Full spec file used verbatim — no section filtering, no summarization. The complete document is the authoritative requirement. Prompt header: `# Functional Specification`.

Use `--description` for rapid prototyping from a short description or when you want the LLM to focus on the high-level intent. Use `--spec` when you have a complete reverse-engineered spec (e.g. `S1-*-spec.md` from `splc describe`) and want the LLM to implement every detail — this is the NeurIPS S7 ablation mode.

Options

Option	Default	Description
`DESCRIPTION`	—	Natural language requirement or path to a `.md` spec file
`--description / -d`	—	Same as positional arg; takes precedence if both given

Option	Default	Description
`--spec`	—	Full spec file (e.g. `S1-spec.md`). Mutually exclusive with `--description`.
`--target / -t`	`python/pocketflow`	Target language/framework
`--adapter`	`ollama`	LLM adapter
`--model / -m`	adapter default	Model override
`--out-dir DIR`	—	**Recommended.** Write code + `README.md` + `test_data.py` to `DIR`
`--output / -o`	stdout	Single-file mode: write code to `FILE` (also writes `<stem>-readme.md`)
`--rag / --no-rag`	`--rag`	Include RAG few-shot examples from the SPL recipe store
`--rag-k`	3	Number of RAG examples (1–10)
`--references`	—	Reference codebase(s): GitHub URL or local path (repeatable)
`--no-readme`	off	Skip generating readme and test data
`--verbose / -v`	off	Print progress and token counts
`--prompt`	off	Print the assembled LLM prompt and exit (no API call)

Examples

```
# Recommended: folder mode — all outputs in one place
spl3 vibe "ReAct research agent" \
  --out-dir ./out --adapter ollama -m gemma3

# From a spec file via --description (extracts Summary/Purpose sections)
spl3 vibe --description S1-agent-spec.md \
  --out-dir ./out/qwen --adapter openrouter -m qwen/qwen3.6-plus
spl3 vibe --description S1-agent-spec.md \
  --out-dir ./out/gemini --adapter openrouter -m google/gemini-3-flash-preview

# Spec-driven (NeurIPS S7 ablation): full spec → code, no .mmd or .spl IR
spl3 vibe --spec S1-agent-spec.md \
  --out-dir ./out/vibe --adapter claude_cli --model claude-sonnet-4-6

# Different target framework
spl3 vibe "RAG pipeline with re-ranking" \
  --target python/langgraph \
  --out-dir ./out --adapter claude_cli

# Preview the assembled prompt without calling the LLM
spl3 vibe --spec S1-agent-spec.md --adapter claude_cli --prompt
```

Output files

Mode	File	Contents
`--out-dir DIR`	`DIR/vibe_output.py`	Complete implementation
	`DIR/README.md`	Setup, usage, expected output
	`DIR/test_data.py`	2–3 realistic test inputs
`--output FILE`	`FILE`	Implementation
	`<stem>-readme.md`	Setup and usage
	`<stem>-test_data.py`	Test inputs

Notes

- **Context window matters**: models with larger context windows (Qwen, Gemini) tend to follow the mandatory 3-section output structure more reliably than models with shorter effective windows. If README or test data are missing, try a larger model.
- **RAG query**: `vibe` passes the raw description directly to the RAG store (unlike `splc compile` which extracts the query from SPL syntax).
- **No manifest**: `vibe` does not write `splc_manifest.json`. Use filename convention (`S7-<recipe>-<adapter>-<model>.py`) to track provenance in experiments.
- **`--no-rag`**: useful when the description is very domain-specific or when isolating the LLM's pretrained knowledge from recipe store examples.

---

17. spl3 code-rag

Manages the Code-RAG vector store that powers `text2spl` and `vibe` few-shot retrieval.

```
spl3 code-rag seed [cookbook/] [--from-specs]
spl3 code-rag stats
```

---

18. spl3 cache — Layer 2 Content Cache

The content cache is a **write-once, input-keyed store** for verified generated sections. Unlike the Layer 1 prompt cache (which is TTL-based and scoped to a single session), the Layer 2 content cache persists across sessions and is invalidated only when its inputs change — not when time passes.

Two cache layers

```
Layer 2 — Content Cache  (spl3/cache)
  Key:  sha256(concept + params + rubric_version + dep_hashes)
  TTL:  none — write-once immutable
  Invalidation: input-driven (CAS), explicit, cascading via dep_graph
```

```
  Scope: cross-session, portable / shareable

      ↓ miss → generate + verify → store

Layer 1 — Prompt Cache  (spl/storage/memory.py)
  Key:  sha256(model + assembled_prompt)
  TTL:  1–24 h (configurable via cache_ttl)
  Scope: per-session / per-workspace

      ↓ miss → LLM call → store
```

A Layer 2 hit skips both the LLM call and the verification loop — the entry is already known-good. On a cold run every concept is generated and stored; on a warm run every concept is a Layer 2 hit and the cost is near zero.

Trust badges — a badge *set* on two axes

Every cached entry carries a **badge set** recording what has been verified about it. Badges live on two orthogonal axes — claim trust attests the mathematical content, exposition trust attests the prose and pedagogy:

Axis	Badge	Attests
claim	`machine_verified`	passed a CAS **instance** check (SymPy/Sage — e.g. `verify_math`, `shape_check`): *this worked example is correct*
claim	`machine_proved`	the **statement** itself is Lean kernel-checked: *the theorem, not one example*
exposition	`ai_reviewed`	passed an `spl3 judge` review; `JudgeResult` stored alongside the entry
exposition	`human_verified`	accepted by a human editor

An entry with no badges is the `machine_generated` baseline — LLM output, nothing verified. Within an axis the order is strict (`machine_proved` outranks `machine_verified`), but **across axes there is no order**: `ai_reviewed` never satisfies a `machine_verified` requirement — prose review says nothing about the math, and vice versa. A section can hold any combination; a kernel-checked statement with confusing exposition still needs review.

`canonical` is **derived, never stored**: the top badge on *both* axes (`machine_proved` + `human_verified`), shown as ★ in the CLI.

Badges only accumulate — promotion adds to the set. The delivery layer filters per axis: `cache.get(concept, …, min_badge="machine_verified")` returns `None` unless the entry holds a claim-axis badge at that rank or higher (`machine_proved` qualifies; `ai_reviewed` does not), ensuring learners never receive unverified content.

Two further provenance columns:

- **`verifier`** — the engine-of-record that actually checked the content (`sympy`, `sage`, `lean`, ...). With fallback declarations like `verifier: "sage|sympy"` in a domain YAML, this records which engine *ran*, never a silent downgrade.
- **`statement`** — for `machine_proved` entries, the kernel-checked Lean proposition. `spl3 cache show` renders the prose and the formal statement **side by side**, so the one link nothing machine-checks — informal↔formal correspondence — can be audited at a glance.

  Migration: databases and exports from the older single-ordinal provenance model (`machine_generated → … → human_verified`) migrate automatically on first open — each legacy tier becomes the badge set attesting only what it attested.

TTL semantics

The content cache never sets a TTL on stored entries — `ttl=None` is always passed internally and callers cannot override it. For the underlying `dd_cache` adapters:

Value	Meaning
`ttl=None`	Never expire (used by Layer 2)
`ttl=0`	Expire immediately — cache bypass, useful in tests
`ttl=N > 0`	Expire after N seconds (used by Layer 1)
`ttl<0`	Treated as immediate expiry by `dd_cache` (no exception raised)

CLI commands

```
# Inspect
spl3 cache list                                # all entries (badges, verifier, hits, …)
spl3 cache list --concept eigenpair            # filter by concept
spl3 cache list --badge machine_proved         # filter by badge ('unbadged' = baseline)
spl3 cache list --format json
spl3 cache show <key>                          # full entry; machine_proved entries rende
                                               # prose ↔ formal Lean statement side by si
spl3 cache stats                               # hit rate, tokens saved, badge breakdown,

# Invalidation
spl3 cache invalidate --concept span           # mark stale; output: list of affected key
spl3 cache invalidate --concept span --cascade   # propagate to all dependents
spl3 cache clear --stale                       # remove all stale-flagged entries
spl3 cache clear --all                         # full wipe (Layer 1 prompt cache unaffect
spl3 cache clear --badge unbadged              # remove unverified entries only

# Portability — team sharing without a live remote store
```

```
spl3 cache export -o cache.tar.gz
spl3 cache import cache.tar.gz              # --merge (default): skip conflicts
spl3 cache import cache.tar.gz --no-merge   # error on key conflict

# Manual promotion — adds a badge to the entry's set (★ shown when canonical)
spl3 cache promote <key> --to human_verified
spl3 cache promote <key> --to machine_proved
```

`spl3 judge <file> --cache-key <key>` is the automated path to the `ai_reviewed` badge — a PASS verdict promotes the entry and stores the `JudgeResult` (§13).

SPL workflow integration

`cache_get` and `cache_put` are registered as stdlib tools — use them in any SPL workflow via `CALL` without any `CREATE TOOL_API` block:

```
-- On-demand textbook: return cached section or generate on miss
WORKFLOW answer_on_demand
    INPUT @concept TEXT

    CALL cache_get(@concept) INTO @section
    EVALUATE @section:
        WHEN miss:
            CALL build_concept_book(@concept) INTO @section
            CALL cache_put(@concept, @section) INTO @cache_key
    APPEND @section TO @lesson
    COMMIT @lesson
END
```

`cache_get` returns the cached content string on a hit, or the sentinel `"miss"` on a cache miss. `cache_put` stores the content and returns the cache key.

`cache_put` accepts the full provenance set — badges (comma-separated), engine-of-record, and the Lean statement backing a `machine_proved` badge:

```
-- With explicit rubric version and domain params
CALL cache_get(@concept, "v2", '{"domain":"linalg"}') INTO @section
CALL cache_put(@concept, @section, badges='machine_verified', rubric_version='v2', para

-- Engine-of-record: which CAS actually verified it
CALL cache_put(@concept, @section, badges='machine_verified', verifier='sage') INTO @ke

-- Proof-grade: kernel-checked Lean statement stored alongside the prose
CALL cache_put(@concept, @report, badges='machine_proved', verifier='lean', statement=@
```

A third stdlib tool, `cache_promote`, adds a badge to an *existing* entry — this is how a Lean post-pass (e.g. recipe 71's `lean_payoffs.spl`) upgrades already-cached sections without regenerating them:

```
CALL cache_promote(@concept, 'machine_proved', statement=@lean_stmt) INTO @badges
```

Cascading invalidation

The cache tracks which concepts depend on which upstream concepts via a `dep_graph` table. Invalidating an upstream concept automatically marks all downstream concepts stale in a single SQL recursive query:

```
# Invalidate "vector" and everything that depends on it
spl3 cache invalidate --concept vector --cascade

# Output:
# Invalidated 3 concept(s): vector, span, eigenpair
```

Stale entries are served with a warning by default. Use `UNLESS STALE` in a `GENERATE` statement (Phase 3) to treat stale entries as misses and force re-generation.

Export / import for team sharing

A pre-warmed cache can be shipped with a textbook deliverable so every reader gets instant responses without re-generating content:

```
# Author: export after full build + verification
spl3 cache export -o linalg-cache-v2.tar.gz

# Reader: import before first use
spl3 cache import linalg-cache-v2.tar.gz
```

The archive contains both the content blobs and the metadata index (badge sets, verifier, statements, hit counts, dep_graph). Importing is idempotent — `--merge` (default) skips any key that already exists locally. Archives exported under the old single-ordinal provenance model import cleanly (tiers are migrated to badge sets automatically).

Default storage paths

File	Contents
`.spl/content_cache.db`	Blob store (dd-cache DiskCache, SQLite)
`.spl/content_meta.db`	Metadata index (badge sets, verifier, statements, dep_graph, hit counts)
`.spl/prompt_cache.db`	Layer 1 prompt cache (unchanged, TTL-based)

---

19. spl3 experiment — Batch Experiment Runner

[To-Test] New in this release (1.1, 1.2).

Two subcommands for running and reporting on NeurIPS-style ablation experiments across multiple recipes, adapters, and models without manually chaining 90+ CLI calls.

`spl3 experiment run`

Runs a full pipeline matrix across `recipes × (adapter, model)` pairs. Skips steps already completed (checkpoint/resume).

```
# Run S1–S6 for two recipes across two adapter/model pairs
spl3 experiment run \
  --recipes self_refine react \
  --adapters claude_cli openrouter \
  --models claude-sonnet-4-6 google/gemini-3-flash-preview \
  --pipeline S1,S2,S3,S4,S5,S6 \
  --spl-root ~/projects/digital-duck/SPL.py/cookbook

# Run ablation steps only (S7–S10) — resumes from completed S1
spl3 experiment run \
  --recipes self_refine \
  --adapters claude_cli \
  --models claude-sonnet-4-6 \
  --pipeline S7,S8,S9,S10

# Dry run — print all commands without executing
spl3 experiment run \
  --recipes self_refine react \
  --adapters claude_cli openrouter \
  --models claude-sonnet-4-6 google/gemini-3-flash-preview \
  --pipeline S1,S2,S3,S4,S5,S6 --dry-run
```

Option	Default	Description
`--recipes`	required	Recipe name(s), repeatable
`--adapters`	required	Adapter name(s), must match `--models` count
`--models`	required	Model ID(s), matched 1:1 with adapters
`--pipeline`	`S1,S2,S3,S4,S5,S6`	Steps to run
`--spl-root DIR`	—	Search for `<recipe>.spl` under this directory
`--spl-paths PATH`	—	Explicit `.spl` paths matching recipes order
`--judge-adapter`	`claude_cli`	Adapter for S6/S9/S10 compare steps
`--judge-model`	`claude-opus-4-6`	Model for compare steps
`--base-dir DIR`	`~/.vibescope/neurips`	Output directory

Option	Default	Description
`--overwrite`	off	Overwrite existing outputs (disables checkpoint)
`--dry-run`	off	Print commands without executing

`spl3 experiment report`

Aggregates compare scores from completed experiments into a ranked leaderboard. Scans `--base-dir` for `S6/S9/S10-*-compare.md` files, extracts Structure/Logic/Quality/Overall scores, computes ΔIR = S6 − S9.

```
# Markdown leaderboard to stdout (default)
spl3 experiment report

# Save as CSV for spreadsheet import
spl3 experiment report --format csv -o leaderboard.csv

# JSON for programmatic processing
spl3 experiment report --format json -o leaderboard.json

# S6 only (no ablation steps run yet)
spl3 experiment report --steps S6
```

Option	Default	Description
`--base-dir DIR`	`~/.vibescope/neurips`	Directory to scan
`--steps STEPS`	`S6,S9,S10`	Compare steps to include
`--format`	`markdown`	`markdown`, `csv`, or `json`
`-o FILE`	stdout	Write report to file

---

20. spl3 migrate — DODA Migration Pipeline `[To-Test]`

`spl3 migrate` automates the full pipeline for porting an existing codebase into SPL and re-compiling to a new target runtime. It wraps four steps into a single command with human checkpoints at the two IR stages (`.mmd` and `.spl`).

Primary use case: migrate PocketFlow cookbook recipes into SPL IR so they can run on any supported runtime (LangGraph, Go, TypeScript, etc.).

```
spl3 migrate ./cookbook/05_self_refine/ \
  --target python/langgraph \
  --name self_refine \
  --adapter claude_cli
```

Steps executed

Step	Command	Output	Checkpoint?
1	`splc describe <source>`	`<name>-spec.md`	—
2	`spl3 text2mmd <name>-spec.md`	`<name>.mmd`	✓ Review Mermaid
3	`spl3 mmd2spl <name>.mmd + spl3 validate`	`<name>.spl`	✓ Review SPL IR
4	`splc compile <name>.spl --target <target>`	`out/<name>/<target>/`	

Optional fidelity compare (skip with `--skip-compare`):

```
splc describe out/<name>/<target>/   # → spec2.md
spl3 compare <name>-spec.md spec2.md --mode llm
```

Options

Flag	Default	Description
`--target`	required	Target runtime (`python/pocketflow`, `python/langgraph`, `go`, `ts`, ...)
`--adapter`	`claude_cli`	LLM adapter for text2mmd and mmd2spl
`--model`	adapter default	Model override
`--judge-adapter`	`claude_cli`	Adapter for fidelity compare synthesis
`--judge-model`	`claude-opus-4-6`	Model for synthesis
`--out-dir`	`./migrate-out`	Root output directory
`--name`	source basename	Name prefix for all artifacts
`--no-rag`	off	Disable Code-RAG context injection
`--skip-compare`	off	Skip fidelity compare step
`--auto`	off	Skip human checkpoints (CI/batch mode)
`--dry-run`	off	Print commands without executing

Checkpoints

By default the pipeline pauses at two points for human review:

1. **After Mermaid generation** — inspect the `.mmd` flow diagram; edit if needed before generating SPL IR.
2. **After SPL IR generation** — inspect the `.spl` file and `spl3 validate` output; edit if needed before compilation.

Use `--auto` to skip both checkpoints for automated pipelines. Use `--dry-run` to print all commands without running anything.

Example: dry run

```
spl3 migrate cookbook/05_self_refine/ \
  --target python/langgraph \
  --name self_refine \
  --dry-run
```

Output shows all 4 steps + fidelity compare commands with artifact paths, so you can verify the plan before execution.

---

21. Full Pipeline S1–S10: IR + Ablation

The ten-step pipeline combines the full IR path (S1–S6) with an ablation baseline (S7–S10) to measure whether the Mermaid + SPL intermediate representations add measurable value over direct vibe coding.

Background: NeurIPS 2026 validation

These steps were designed for the NeurIPS 2026 paper *"Beyond Vibe Coding: Intent Invariance and Structured Prompt Language"* and executed as 15 experiments (5 recipes × 3 model tiers). In doing so, every SPL.py command was exercised end-to-end on real PocketFlow workflows — making these experiments the most comprehensive integration validation of the toolchain to date. **The steps are not just a research protocol; each one is a useful standalone feature.**

The ten steps at a glance

Step	Command	Output	Standalone value
S1	`spl3 splc describe --include-docs`	`S1-*-1-spec.md`	Reverse-engineer any codebase into a spec
S2	`spl3 text2mmd`	`S2-*.mmd`	Visualise workflow topology from a spec
S3	`spl3 mmd2spl`	`S3-*.spl`	Convert a diagram to executable SPL
S3✓	`spl3 spl2mmd`	`S3-*.mmd`	**Visual validation:** re-render S3 SPL as diagram, compare with S2
S4	`spl3 splc compile --llm`	`S4-*.py`	Compile SPL to a target framework
S5	`spl3 splc describe`	`S5-*-2-spec.md`	Spec the compiled artifact
S6	`spl3 compare S1 S5`	`S6-*-spec-diff.md`	Measure round-trip intent fidelity (ΔS)

Step	Command	Output	Standalone value
S7	`spl3 vibe --out-dir`	`vibe/` folder	One-shot prototype from spec (bypass IR)
S8	`spl3 splc describe`	`S8-*-3-spec.md`	Spec the vibe-generated artifact
S9	`spl3 compare S1 S8`	`S9-*-vibe-diff.md`	Measure vibe round-trip fidelity
S10	`spl3 compare S6 S9`	`S10-*-ablation.md`	**ΔIR = S6 − S9** (IR value-add)

Human Checkpoint Protocol

Every `[Human Checkpoint]` in S1–S10 follows the same three-step protocol:

```
1. REVIEW   — inspect the generated artifact
2. RUN      — execute/test with real inputs; work with AI assistant to fix all issues
3. DOCUMENT — record issues and fixes in notes.md before proceeding
```

Checkpoints occur after S1 (spec quality), S2 (diagram topology), S3 (SPL validation + run), S3✓ (visual diff of re-rendered diagram vs. S2), S4 (compiled code test), S6 (review score), S7 (vibe output run), S10 (ablation verdict). Steps S5, S8, S9 are fully automated.

Phase 1 — Full IR pipeline (S1→S6)

```
existing code / README
    → S1: spl3 splc describe --include-docs    spec (ground truth)
    → S2: spl3 text2mmd                        Mermaid diagram
          ⚠  [Human] verify topology
    → S3: spl3 mmd2spl                         SPL workflow
       spl3 spl2mmd S3-*.spl           re-render as diagram for visual diff vs S2
          ⚠  [Human] spl3 validate + spl3 run
    → S4: spl3 splc compile --llm              compiled code
          ⚠  [Human] run with test inputs
    → S5: spl3 splc describe                   round-trip spec
    → S6: spl3 compare S1 S5                   ΔS score
          ⚠  [Human] review score; trace drift to S2 or S3
```

Phase 2 — Ablation baseline (S7→S10)

```
    → S7: spl3 vibe --out-dir                  vibe-coded folder
          ⚠  [Human] run with same test inputs as S4
    → S8: spl3 splc describe <vibe-folder>     vibe spec
    → S9: spl3 compare S1 S8                   vibe ΔS score
    → S10: spl3 compare S6 S9                  ΔIR = S6 − S9
           ⚠  [Human] review ablation report; aggregate across runs
```

Canonical command sequence (R1-agent / claude example)

```
export RECIPE=agent  ADAPTER=claude_cli  MODEL_ID=claude-sonnet-4-6  MODEL=sonnet
export OUT=~/projects/digital-duck/SPL.py/NeurIPS-26-lab/R1-$RECIPE/tests/$ADAPTER/$MOD
export SRC=~/projects/digital-duck/SPL.py/NeurIPS-26-lab/R1-$RECIPE/src/pocketflow-$REC

# S1 — describe original code
spl3 splc describe $SRC --include-docs --adapter $ADAPTER --model $MODEL_ID \
  -o $OUT/S1-$RECIPE-$ADAPTER-$MODEL-1-spec.md

# S2 — Mermaid diagram  *** HUMAN CHECKPOINT: verify topology ***
spl3 text2mmd $OUT/S1-$RECIPE-$ADAPTER-$MODEL-1-spec.md \
  --adapter $ADAPTER --model $MODEL_ID -o $OUT/S2-$RECIPE-$ADAPTER-$MODEL.mmd

# S3 — SPL workflow
spl3 mmd2spl $OUT/S2-$RECIPE-$ADAPTER-$MODEL.mmd \
  --adapter $ADAPTER --model $MODEL_ID -o $OUT/S3-$RECIPE-$ADAPTER-$MODEL.spl
spl3 validate $OUT/S3-$RECIPE-$ADAPTER-$MODEL.spl

# S3✓ — visual validation: re-render S3 SPL as diagram, compare with S2
spl3 spl2mmd $OUT/S3-$RECIPE-$ADAPTER-$MODEL.spl \
  --out-dir $OUT/S3-check/ --no-preview --save-pdf

# S4 — compile to Python/PocketFlow
spl3 splc compile $OUT/S3-$RECIPE-$ADAPTER-$MODEL.spl \
  --lang python/pocketflow --llm --adapter $ADAPTER --model $MODEL_ID \
  --out-dir $OUT/targets/python_pocketflow --overwrite

# S5 — describe compiled code
spl3 splc describe $OUT/targets/python_pocketflow/S4-$RECIPE-$ADAPTER-$MODEL.py \
  --adapter $ADAPTER --model $MODEL_ID -o $OUT/S5-$RECIPE-$ADAPTER-$MODEL-2-spec.md

# S6 — round-trip score (judge fixed at claude-opus-4-6)
spl3 compare $OUT/S1-$RECIPE-$ADAPTER-$MODEL-1-spec.md \
  $OUT/S5-$RECIPE-$ADAPTER-$MODEL-2-spec.md \
  --adapter claude_cli --model claude-opus-4-6 \
  -o $OUT/S6-$RECIPE-$ADAPTER-$MODEL-spec-diff.md

# S7 — vibe-coded baseline  *** HUMAN CHECKPOINT: run generated code ***
# --spec uses the full S1 spec verbatim (no section filtering) — same input as IR pipel
mkdir -p $OUT/vibe/python_pocketflow
spl3 vibe --spec $OUT/S1-$RECIPE-$ADAPTER-$MODEL-1-spec.md \
  --target python/pocketflow --adapter $ADAPTER --model $MODEL_ID \
  --out-dir $OUT/vibe/python_pocketflow

# S8 — describe vibe output
spl3 splc describe $OUT/vibe/python_pocketflow \
```

```
  --adapter $ADAPTER --model $MODEL_ID \
  -o $OUT/S8-$RECIPE-$ADAPTER-$MODEL-3-spec.md

# S9 — vibe round-trip score
spl3 compare $OUT/S1-$RECIPE-$ADAPTER-$MODEL-1-spec.md \
  $OUT/S8-$RECIPE-$ADAPTER-$MODEL-3-spec.md \
  --adapter claude_cli --model claude-opus-4-6 \
  -o $OUT/S9-$RECIPE-$ADAPTER-$MODEL-vibe-diff.md

# S10 — ablation: IR value-add = S6 score − S9 score
spl3 compare $OUT/S6-$RECIPE-$ADAPTER-$MODEL-spec-diff.md \
  $OUT/S9-$RECIPE-$ADAPTER-$MODEL-vibe-diff.md \
  --adapter claude_cli --model claude-opus-4-6 \
  -o $OUT/S10-$RECIPE-$ADAPTER-$MODEL-ablation.md
```

When to use each path

Situation	Recommendation
Rapid prototype — explore a new framework or idea	S7 (`spl3 vibe`)
Multi-model comparison — same spec, different models	S7 × N models
Need traceable, validated, auditable artifacts	S1→S6 (full IR)
Quantify IR value-add for a specific workflow	S1→S10 (both paths)
Already have a `.spl` file, new target	S4 only (`splc compile`)
Already have compiled code, need spec	S5 only (`splc describe`)
Visual audit of any `.spl` file (no LLM)	`spl3 spl2mmd`
Publication-quality diagram from `.spl`	`spl3 spl2mmd --save-pdf`

---

22. Debugging LLM Prompts

All LLM-powered commands support the `--prompt` flag. It prints the full assembled prompt — including RAG hits and reference context — then exits without calling the API.

```
# Preview the vibe prompt
spl3 vibe "my workflow" --adapter claude_cli --prompt

# Preview the splc compile prompt
spl3 splc compile agent.spl --lang go --llm --prompt

# Preview the Mermaid generation prompt
spl3 text2mmd "my workflow" --prompt

# Preview the mmd2spl prompt (requires --adapter)
spl3 mmd2spl workflow.mmd --adapter claude_cli --prompt
```

The output includes prompt length in characters and approximate token count.

---

23. Claude Code `/spl3` Skill

The `/spl3` skill lets you invoke any `spl3` command directly from the Claude Code chat prompt — no terminal switching required.

Install

The skill ships inside the `spl-llm` package. Run once after `pip install`:

```
spl3 install-skill            # global install (~/.claude)
spl3 install-skill --local    # project-local install (./.claude)
spl3 install-skill --dry-run  # preview what will change
```

This copies `SKILL.md` to `~/.claude/skills/spl3/` and registers the skill in `~/.claude/CLAUDE.md`. The command is idempotent — safe to re-run after upgrading `spl-llm`.

Verify

Open a new Claude Code session (or type `/clear`), then:

```
/spl3 --help
```

Claude prints the full `spl3` command table, alphabetically sorted.

Usage inside Claude Code

Prefix any `spl3` command with `/spl3`:

What you type	What runs
`/spl3 --help`	`spl3 --help`
`/spl3 run hello.spl --adapter ollama`	`spl3 run hello.spl --adapter ollama`
`/spl3 text2spl "a parallel news digest"`	`spl3 text2spl --description "a parallel news digest"`
`/spl3 validate my_workflow.spl`	`spl3 validate my_workflow.spl`
`/spl3 splc compile agent.spl --lang go`	`spl3 splc compile agent.spl --lang go`
`/spl3 show`	`spl3 show`

Claude also assists with authoring: if you ask it to write or edit an `.spl` workflow it automatically validates after each edit and suggests `spl3 explain` before the first run.

How it works

```
/spl3 run hello.spl
       │
       ▼  Claude Code reads ~/.claude/skills/spl3/SKILL.md
       │
       ▼  Claude follows the skill instructions
       │
       ▼  Bash tool executes: spl3 run hello.spl
       │
       ▼  Output shown in the session
```

The skill is pure Markdown — no Python code, no entry-point wiring.

Uninstall

```
rm -rf ~/.claude/skills/spl3
# then remove the 3-line "# spl3" block from ~/.claude/CLAUDE.md
```

Developer reference

See `docs/DEV/spl3-plugin.md` for the full plugin architecture, project-scoped install, and how to extend the skill with new commands.

---

24. Command reference

```
spl3 validate <file.spl> [file.spl ...]        # syntax check
         [--semantic/--no-semantic]         # semantic lint (default: on)  [To-
Test]
          [--strict]                     # warnings → errors         [To-Test]
spl3 run <file.spl> [--adapter] [--model] [-p key=val] [--log-prompts DIR]
            [--tools FILE]                          # load @spl_tool functions
            [--kernel]                             # persistent IPython kernel
          [--kernel-name NAME]               # kernel spec, e.g. sagemath (implies -
-kernel)
              [--kernel-scope session|workflow]        # default: session
              [--kernel-timeout FLOAT]                 # default: 60.0 s
           [--persistence sqlite|postgres|dbos]    # durable execution (§4.3)
           [--workflow-id ID]                 # run ID; auto-UUID when omitted

spl3 workflow list [--backend sqlite|postgres|dbos] [--status running|complete|error]
spl3 workflow status --workflow-id ID [--backend sqlite|postgres|dbos]
spl3 workflow send-event --workflow-id ID --key KEY --value VALUE [--
backend ...]
spl3 workflow resume <file.spl> --workflow-id ID [--adapter] [--model] [--
backend ...]
```

```
spl3 describe <file.spl | folder/> [--adapter] [--model] [--prompt]
spl3 text2spl "<description>" [--adapter] [--mode auto|prompt|workflow] [--
prompt]
spl3 text2mmd "<description>|<file.md>" [--adapter] [--style flowchart|graph|sequence] [-
-prompt]
          # file input: extracts ## Summary / ### 1. Purpose sections, or LLM-
summarizes
spl3 img2mmd <image_path> [--adapter] [--model] [-o FILE] [--out-dir DIR]
spl3 img2text <image_path> [--adapter] [--model] [-o FILE] [--out-dir DIR]
spl3 spl2mmd <file.spl> [file.spl ...]
              [--out-dir DIR]
              [--preview/--no-preview]      # default: on
              [--save-html/--no-save-html]  # default: on
              [--save-markdown/--no-save-markdown]  # default: on
              [--save-spl/--no-save-spl]    # default: on
              [--save-png]                  # default: off
            [--save-pdf]                # default: off; needs mmdc or Chrome
spl3 mmd2spl <file.mmd> [--adapter] [--template workflow|function] [--prompt]
spl3 compare <f1> <f2>
                     [--mode  llm|git-diff|vector|bert-score|ged|vision|ast-
diff|structural|rouge]  # rouge: [To-Test]
              [--adapter NAME] [--model MODEL]
              [--adapter-embed NAME] [--adapter-synthesis NAME]
              [--format markdown|json|text|html] [-o FILE]
              [--focus all|structure|logic|quality|syntax|spl]
              [--diff-style unified|context|side-by-side]
              [--synthesize/--no-synthesize]
              [--prompt]
spl3 judge <file>
                        [--criteria  spl-compliance|correctness|clarity|ai-
review|FILE.yaml]
           [--llm ADAPTER:MODEL]         # repeat for panel mode; wins over -
-adapter/--model
              [--adapter NAME] [--model MODEL]  # legacy; used only when --
llm not given
          [--aggregation majority|confidence_weighted|unanimous] # panel only
                       [--swap-check]                                  #  re-
run with reversed criteria; flags inconsistency
              [--format markdown|json|text] [-o FILE]
              [--prompt]
          [--cache-key KEY]          # on PASS: add ai_reviewed badge to cache entry
spl3 vibe "<description>" [--target LANG] [--adapter] [--model]
          [--out-dir DIR] [-o FILE]
           [--rag/--no-rag] [--rag-k N] [--references URL] [--verbose] [--
prompt]
# or spec-driven (full spec, no section filtering — NeurIPS S7 ablation mode):
spl3 vibe --spec <SPEC_FILE> [--target LANG] [--adapter] [--model]
          [--out-dir DIR] [-o FILE]
```

```
          [--rag/--no-rag] [--rag-k N] [--verbose] [--prompt]

spl3 splc compile <file.spl> --lang <target> [--llm] [--adapter] [--rag-k N]
                 [--references URL] [--out-dir DIR] [--overwrite] [--prompt]
            [--kernel-name NAME]      # domain targets: emitted .ipynb declares this kern
spl3 splc describe <impl.py | folder/> [--lang LABEL] [--adapter] [--spec-
dir DIR]
                   [-o FILE] [--include-docs] [--prompt]

spl3 code-rag seed [cookbook/] [--from-specs]
spl3 code-rag stats

# Layer 2 content cache
spl3 cache list [--concept NAME] [--badge BADGE|unbadged] [--format table|json]
spl3 cache show <key>                       # machine_proved: prose ↔ Lean statement side b
spl3 cache stats                           # badge breakdown + ★ canonical count
spl3 cache invalidate --concept NAME [--cascade/--no-cascade]
spl3 cache clear --stale                        # remove stale-flagged entries
spl3 cache clear --all                                  # full wipe
spl3 cache clear --badge BADGE|unbadged          # remove by badge ('unbadged' = unverifie
spl3 cache export -o FILE.tar.gz
spl3 cache import FILE.tar.gz [--merge/--no-merge]   # default: merge (skip conflicts)
spl3 cache promote <key> --to BADGE             # add a badge; claim axis: machine_verified
                                     # exposition axis: ai_reviewed<human_verified; axes i
                                     # ★ canonical = top badge on both axes (derived, never

# Batch experiment runner  [To-Test]
spl3 experiment run
              --recipes RECIPE [RECIPE ...]          # recipe name(s)
              --adapters ADAPTER [ADAPTER ...]       # adapter name(s)
          --models MODEL [MODEL ...]         # model IDs (matched 1:1 with adapters)
              [--pipeline S1,S2,S3,S4,S5,S6]        # default: S1–S6
            [--spl-root DIR]                    # search dir for <recipe>.spl
              [--spl-paths PATH [PATH ...]]          # explicit .spl paths
             [--judge-adapter claude_cli]           # adapter for S6/S9/S10
           [--judge-model claude-opus-4-6]       # model for compare steps
              [--base-dir ~/.vibescope/neurips]      # output directory
              [--overwrite] [--dry-run]

spl3 experiment report                     # leaderboard from completed runs  [To-
Test]
              [--base-dir ~/.vibescope/neurips]
              [--steps S6,S9,S10]
              [--format markdown|csv|json]
              [-o FILE]

# DODA migration pipeline  [To-Test]
spl3 migrate <source>
```

```
          --target <runtime>                # required: python/langgraph, go, ts, …
               [--adapter claude_cli]
               [--model MODEL]
               [--judge-adapter claude_cli]
               [--judge-model claude-opus-4-6]
               [--out-dir ./migrate-out]
             [--name NAME]                          # default: source basename
               [--no-rag]                                 # disable Code-RAG
              [--skip-compare]                        # skip fidelity compare
             [--auto]                                # skip human checkpoints
              [--dry-run]                               # print commands only

# Claude Code skill
spl3 install-skill            # install /spl3 skill to ~/.claude (global)
             [--local]        # install to ./.claude (project-scoped)
             [--dry-run]      # preview changes without writing files
```

---

25. PocketFlow Cookbook Recipes

`cookbook-pocketflow/` contains **41 SPL recipes** ported from the PocketFlow cookbook, covering the full range of agentic workflow patterns. Each recipe is a self-contained `.spl` + `tools.spl` pair that runs with any SPL adapter — the same `.spl` works on `ollama`, `claude_cli`, `openrouter`, and `momagrid` without modification.

Location

```
cookbook-pocketflow/
  000_hello_world/
  001_qa/
  ...
  064_visualization/
  readme-migration.md    ← full recipe catalog and migration notes
```

Recipe catalog by phase

Phase	Recipes	Patterns demonstrated
1 — Core	000–009	Hello world, QA, structured output, chat history, LLM orchestration
2 — Agentic	010–019	ReAct agents, decision making, multi-agent handoff
3 — Memory & storage	020–039	Memory agents, multi-modal, web search, async parallel
4 — Advanced agents	040–042	Coding agents, skill agents, Node.js upgrade agent

Phase	Recipes	Patterns demonstrated
5 — Specialized	050–064	Batch processing, map-reduce, RAG, crawlers, PDF vision

Quick-start examples

```
# Hello world (no tools needed)
spl3 run cookbook-pocketflow/000_hello_world/hello_world.spl \
  --adapter ollama -m gemma3

# ReAct agent with tool use
spl3 run cookbook-pocketflow/010_agent/agent.spl \
  --adapter claude_cli -m claude-sonnet-4-6 \
  -p task="find the population of Tokyo"

# Multi-agent supervisor
spl3 run cookbook-pocketflow/013_multi_agent/multi_agent.spl \
  --adapter ollama -m gemma3 \
  -p topic="climate change solutions"

# Async parallel batch (N tasks run concurrently)
spl3 run cookbook-pocketflow/036_async_parallel_batch/async_parallel_batch.spl \
  --adapter ollama -m gemma3

# Coding agent (writes and runs Python)
spl3 run cookbook-pocketflow/040_coding_agent/coding_agent.spl \
  --adapter claude_cli -m claude-sonnet-4-6 \
  -p task="write a function to compute fibonacci numbers"

# Node.js upgrade agent (framework-aware, template-guided)
spl3 run cookbook-pocketflow/042_nodejs_upgrade_agent/nodejs_upgrade_agent.spl \
  --adapter claude_cli \
  -p project_dir="./my-next-app" \
  -p target_node="20"
```

Agentic workflow pattern coverage

The PocketFlow recipes extend SPL's built-in cookbook with 21 distinct agentic patterns:

Pattern	SPL construct	Representative recipe
Single LLM call	`GENERATE`	000_hello_world, 001_qa
Structured output	`GENERATE` + `CALL parse_field`	003_structured_output

Pattern	SPL construct	Representative recipe
Chat history	`@history :=` accumulation	004_chat_history
LLM cascade / orchestration	chained `GENERATE`	005_llm_orchestration
ReAct agent	`WHILE` + `EVALUATE` + `CALL`	010_agent
Multi-agent handoff	`CALL` sub-workflows	013_multi_agent
Supervisor pattern	`EVALUATE` routing + `CALL`	015_multi_agent_supervisor
HITL approval gate	`CALL wait_for_approval`	any `--persistence` workflow
Tool use	`CREATE TOOL_API` + `CALL`	002_tool_use, 040_coding_agent
Memory (episodic)	`CALL` store/retrieve tools	022_memory
Multi-modal (image)	`GENERATE` with `IMAGE` param	024_multi_modal
Web search	`CREATE TOOL_API` (search)	030_web_search
Async parallel	`CALL PARALLEL … END`	036_async_parallel_batch
Batch / map-reduce	`WHILE` + `CALL PARALLEL`	050_batch, 053_map_reduce
RAG pipeline	`CALL embed` + `CALL retrieve`	055_rag
PDF/vision extraction	`GENERATE` with `IMAGE` param	061_pdf_vision
Web crawler	`WHILE` + `CALL fetch`	058_tool_crawler
Code generation + execution	`CALL run_python` + `ASSERT`	040_coding_agent
Self-refinement	`WHILE` quality gate	005_self_refine (see main cookbook)
Evaluate / branch	`EVALUATE … WHEN … ELSE`	033_streaming_output
Exception handling	`EXCEPTION WHEN … THEN`	039_async_streaming

SPL patterns used across PocketFlow recipes

```
-- Tool defined inline (no tools.py needed for simple tools)
CREATE TOOL_API search_web(query TEXT) RETURNS TEXT AS PYTHON $
import requests
def search_web(query):
    ...
$;

-- Parallel fan-out over a list of tasks
```

```
CALL PARALLEL
  CALL summarize(@task1) INTO @sum1;
  CALL summarize(@task2) INTO @sum2;
  CALL summarize(@task3) INTO @sum3;
END;

-- Crash-safe long-running agent
spl3 run agent.spl --adapter claude_cli \
  --persistence sqlite --workflow-id agent-run-001
```

Notes for recipe authors

- All `CREATE TOOL_API` Python bodies use `'single'` quotes — never `''double-single''` (PostgreSQL SQL style is invalid Python and will fail at load time).
- Recipes that need `pdf2image` or `playwright` document the `pip install` in their `readme.md`; those packages are not bundled with `spl-llm`.
- See `cookbook-pocketflow/readme-migration.md` for the complete recipe status, pitfall list, and pattern coverage taxonomy.

Running the full PocketFlow suite

```
# All active recipes, local Ollama
python cookbook/run_all.py \
  --catalog cookbook-pocketflow/cookbook_catalog.json \
  --adapter ollama --model gemma3

# Specific recipes by ID
python cookbook/run_all.py \
  --catalog cookbook-pocketflow/cookbook_catalog.json \
  --ids 010,013,015,040

# Against Momagrid distributed grid
export MOMAGRID_HUB_URL=http://192.168.0.184:9000
python cookbook/run_all.py \
  --catalog cookbook-pocketflow/cookbook_catalog.json \
  --adapter momagrid --workers 10
```